Report on Objective 1: PF Layer Buffer Manager Enhancements

1. Objective

The primary objective was to enhance the Paged File (PF) layer's buffer manager. The initial implementation provided a fixed-size buffer (PF_MAX_BUFS), a non-selectable LRU eviction policy, and a basic dirty page handling mechanism.
The goals were to:

1. Implement a configurable buffer pool size.
2. Introduce selectable page replacement strategies (LRU and MRU).
3. Integrate a statistics collection module for performance monitoring.
4. Analyze the performance trade-offs between the implemented strategies.

2. Implementation Summary

Modifications were concentrated within the PF layer files: pf.c, buf.c, pf.h, and pftypes.h.

2.1. Configurable Parameters

The PF_Init function signature in pf.h was modified to accept configuration parameters:
void PF_Init(int num_buffers, int replacement_strategy);
To support this, the hardcoded #define PF_MAX_BUFS was removed from pftypes.h, and strategy constants (STRATEGY_LRU, STRATEGY_MRU) were added. In buf.c, static global variables (PFmaxbpage, PFstrategy) were introduced to store these runtime values, and the buffer allocation logic was updated to use PFmaxbpage.

2.2. Page Replacement Logic

The existing implementation utilized a doubly linked list, implicitly functioning as an LRU policy by evicting from the tail (PFlastbpage).
We modified the victim selection logic in PFbufInternalAlloc to check the PFstrategy variable.

- **LRU:** The original behavior (evict from PFlastbpage) is maintained.
- **MRU:** A new logic path was added to scan from the head of the list (PFfirstbpage) and evict the *most* recently used non-fixed page.

2.3. Dirty Flag

The existing dirty flag (dirty : 1;) and write-back policy were retained without modification. Dirty pages are correctly written to disk only upon eviction or via PF_CloseFile, ensuring data persistence.

2.4. Statistics Collection

A statistics module was integrated into buf.c to provide performance metrics.

1. **Counters:** Five static global counters were added (e.g., g_num_logical_reads, g_num_buffer_hits, g_num_physical_reads).
2. **Instrumentation:** Key functions were instrumented:
    - PFbufGet: Increments g_num_logical_reads on every call. It increments g_num_buffer_hits on a cache hit or g_num_physical_reads on a miss.
    - PFbufAlloc: Increments g_num_logical_writes.
    - PFbufInternalAlloc / PFbufReleaseFile: Increment g_num_physical_writes when a dirty page is flushed.
3. Verification: The implementation correctly satisfies the identity:
   Logical Reads = Buffer Hits + Physical Reads

3. Performance Analysis

A test program (buffertest.c) was developed to simulate a mixed workload, consisting of:

1. **Hot Set (5 pages):** Frequent, random access (80% of requests).
2. **Cold Set (50 pages):** Sequential access (20% of requests).
3. **Buffer Size (20 pages):** Sized to hold the hot set but not the entire cold set.

3.1. Analysis of Results

The test results demonstrate the classic behaviors of LRU and MRU under a workload with high locality of reference.

- **LRU Strategy (Hit Rate: 79.92%):** Performed excellently. The 80% of requests targeting the "hot set" ensured these 5 pages were constantly marked as "recently used," keeping them at the head of the buffer list. The "cold scan" pages were correctly identified as "least recently used" and placed at the tail, making them the first victims for eviction. This protected the high-utility hot set, resulting in a high hit rate.
- **MRU Strategy (Hit Rate: 42.45%):** Performed poorly. Because the "hot set" pages were accessed most frequently, they were always at the head of the list. The MRU policy evicts from the head, meaning it was *consistently evicting the most valuable pages* (the hot set). This is the worst-case scenario for a workload with high locality, leading to a low hit rate.

4. Conclusion

All requirements for Objective 1 were met. The buffer manager is now configurable by size and replacement strategy. The statistical analysis demonstrated that for a mixed workload with a clear "hot set" (high locality of reference), the **LRU strategy is vastly superior** as it correctly identifies and protects high-utility pages.
The MRU strategy, while noted as useful for pure sequential scans, was shown to be highly detrimental in a mixed workload, as it preferentially evicts the most valuable cached data.